

자동화는 코드가 아니라 설계다

실전 영상 공장을 해부해, 내 자동화 설계도를 그린다.

P Pipeline · 쪼갬다

I I/O · 정한다

L Layer · 나눈다

O Oversight · 지킨다

T Trace · 남긴다

자동화의 진짜 어려움은 설계다

"툴만 붙이면 되겠지" — 현실은 대개 세 번째 단계에서 무너진다.

흔한 착각 X

- 좋은 툴(자동화 앱) 하나 붙이면 알아서 된다
- AI가 똑똑하니 "다 해줘" 한 번에 시키면 된다
- 결과 → 앞부분은 되는데 3번째 단계에서 뒤엎겨 멈춘다

진짜 핵심 ✓

- 일을 어떻게 쪼개고, 단계끼리 뭘 주고받게 할지
- = 툴이 아니라 설계의 문제
- 5차의 말: "AI가 못하는 게 아니라 전달이 부족한 것"

그래서 오늘은 툴 사용법이 아니라 — 설계하는 눈을 기른다.

남의 공장을 해부해, 내 설계도를 그린다

코드는 안 짭니다. 오늘의 결과물은 '내 자동화 설계도' 한 장.

01 · 견학

공장을 통째로

제품 정보 → 광고 영상. 실제 돌아가는 걸 본다.

02 · 해부

PILOT 5칼

다섯 원칙으로 뜯어 설계 원리를 뽑는다.

03 · 설계

내 설계도 1장

내 업무를 PILOT 5칸으로 그린다.

만드는 건 그다음(6·7차 방식) — 오늘은 **설계가 먼저다**.

제품 정보 넣으면 광고 영상이 나온다

입력 = 제품 정보 → 출력 = 한국어 음성이 깔린 광고 영상.

완성 영상 재생 — 오늘 유일하게 '원본 그대로' 보여주는 것. 이해가 아니라 감탄 포인트.

사람이 편집한 게 아니라,
단계를 밟는 공장에서 나왔다.

여덟 칸의 흐름

복잡해 보여도, 지금부터 PILOT 다섯 칼로 해부하면 단순해진다.



각 칸은 **판단·글쓰기**(Claude)거나 **그림·영상 생성**(도구). 섞이지 않는다.

공장 옆에 — 내 업무도 같이 본다

거창한 영상 공장만이 아니라, 여러분이 바로 쓸 작은 예시를 나란히 놓고 갑니다.

예시 A · 영상 공장 (해부 대상)

- 제품 정보 → 광고 영상 8단계
- 규모가 크지만 원리가 또렷하게 보임

예시 B · 주간 리포트 (내 것)

- 흠어진 기록 → 주간 리포트 초안
- 누구나 당장 자동화할 만한 크기

원칙마다 **둘 다**로 확인 — "큰 것도, 작은 내 업무도 똑같이 적용된다".

한 방에? vs 여러 칸으로

한 번에 다 시키면 어디서 틀어졌는지 모른다. 쪼개면 틀어진 칸만 다시.

한 방에 시키기 x

제품 → ??? → 영상

대본이 문제인지 그림이 문제인지 뒤영킴 · 고칠 곳을 못 찾음

단계로 쪼개기 ✓

여러 칸으로 분해

틀어진 칸만 다시 · 칸마다 전문화 · 흐름이 눈에 보임

"AI가 똑똑하니 한 번에" — 큰 일일수록 **정반대**. 똑똑할수록 **쪼개서** 시켜야 한다.

"다 해줘"라고 하면 생기는 일

실제로 한 방에 시켜본 결과 — 자동화가 아니라 도박이 된다.

증상 1

앞은 그럴듯한데 뒤로 갈수록 **대본과 그림이 따로 논다** — 어느 단계가 원인인지 알 수 없다.

증상 2

한 군데만 고치고 싶는데 **전체를 다시** 돌려야 한다 → 시간·비용 폭발.

증상 3

될 때도 있고 안 될 때도 있다 — **재현이 안 된다**. 자동화의 생명인 '똑같이 나옴'이 깨진다.

교훈: 큰 일을 한 프롬프트에 욕여넣지 말 것. **쪼개는 순간** 자동화가 된다.

좋은 단계의 세 조건

01

하나의 책임

한 칸은 한 일만. '분석'과 '대본'을 한 칸에 넣지 않는다.

02

끝이 분명

끝나면 딱 떨어지는 결과물. 대본 한 편, 그림 여섯 장.

03

다음 칸이 받는다

앞 칸의 결과가 뒤 칸의 재료가 된다.

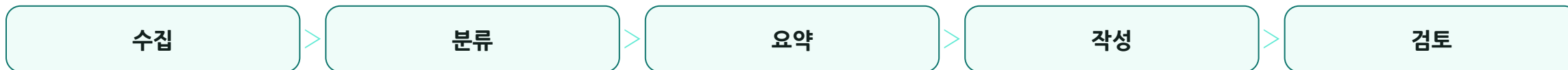
개수의 감: 보통 **3~7단계**. 2개면 안 쪼갬 것, 20개면 관리가 더 복잡 — "한 칸=한 책임"이 기준.

같은 원리, 두 크기로 쪼개기

예시 A · 영상 공장



예시 B · 주간 리포트



크기는 달라도 규칙은 같다 — 한 칸=한 책임, 끝이 분명.

단계를 **계약**으로 잇는다

칸 사이는 '말'이 아니라 정해진 양식의 결과물로. 각 칸의 입력→출력을 못 박는다.

대본 단계 — 입출력 계약

들어가는 것	영상 의도 (누구에게 · 어떤 느낌)
나오는 것	컷별 대사 + 장면 묘사

"알아서 잘 넘겨줘"가 아니라 — **뭘 받고 뭘 내놓을지**를 미리 정한다.

양식이 없으면 중간에 멈춘다

계약 없음 ✕

- 대본 칸이 어떤 날은 대사만, 어떤 날은 줄거리만 내놓음
- 다음 칸(그림)이 뭘 받을지 몰라 깨진다
- 사람이 매번 손봐야 함 → 자동화 아님

계약 있음 ✓

- "항상 컷별 대사+장면 묘사" 양식 고정
- 다음 칸은 믿고 그대로 받아 진행
- 사람 개입 없이 흐른다

자동화가 **중간에 멈추는** 대부분의 이유 = 단계 사이 **양식이 안 정해져서**.

계약이 있으면 — 갈아끼울 수 있다

따로따로 가능

- 칸을 따로 고치고, 따로 테스트
- 앞 칸이 어떻게 만들었는지 몰라도 됨
- 결과물 모양만 약속되면 OK

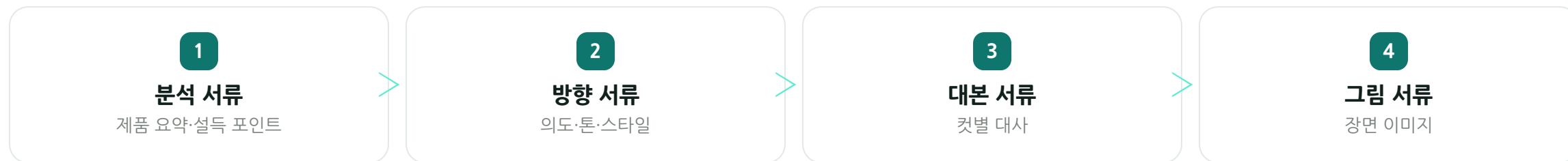
실행자 교체

- 대본을 사람이 쓰든 AI가 쓰든
- "의도 받아 대사 낸다" 계약만 지키면
- 뒤 칸은 신경 안 쓴다

2차 강의의 **입출력 계약(포·필·범·메)** — 자동화에선 '단계의 계약'으로 그대로 온다.

단계 사이 결과물 = 서류철

각 칸이 일한 결과를 '서류(파일)'로 만들어 다음 칸 책상에 올려둔다. 이 서류를 **아티팩트**라 부른다.



서류만 있으면 **누가 이어받아도** 일이 굴러간다 — 사람이 바뀌어도, AI로 바뀌어도.

가로로 쪼개고, 세로로 나눈다

① 가로 → 쪼갬다 · Pipeline (시간 순서)



② 세로 ↓ 나눈다 · Layer (성격 분리)

- 지식** 안 변하는 규칙·자료
- 스펙** 각 칸의 계약서
- 지휘자** 순서대로 시킴
- 창구** 요청 넣는 곳

두 축으로 분해한다.

가로(P)는 시간, 세로(L)는 성격. 이 두 축이 오늘 강의의 뼈대 — 6차 '파일=한 책임'이 시스템 규모로 커진 것.

각 층은 무엇을 맡나

지식

안 변하는 **재료·규칙**. 제품 지식, 제작 지침, 스타일 템플릿. "무엇을 아는가"만 담고 실행은 모름.

스펙

각 칸의 **계약서**. 입력→출력·규칙. 언어중립 — 누가 실행해도 똑같이 읽힘.

지휘자

칸들을 **순서대로 시키고 상태를 관리**. 창·의·생성은 안 함. '순서·상태·호출'만.

창구

사람이 **요청을 넣고 결과를 받는 곳**. 웹 화면이든 채팅이든.

규칙을 바꾸려면 **지식층**만, 순서를 바꾸려면 **지휘자층**만 — 서로 안 건드린다.

얇은 지휘자 — 갈아끼운다

지휘자는 '순서·상태·호출'만. 창의로 생성도 안 한다. 그래서 통째로 교체가 된다.

오늘 — 지휘자 = Cowork

사람이 트리거

세션에서 커서 8단계를 돌린다

내일 — 지휘자 = Python + API

사람 없이 무인

지식·계약은 그대로, 지휘자만 교체

지휘자가 **얇아서** 갈아끼울 수 있다 — 이게 잘 나눈 설계의 진짜 힘.

무엇을 · 어떻게 · 어디서 를 가르다

WHAT · 지식

무엇을 아는가

규칙·자료. 바뀌도 실행 코드는 안 건드릴.

HOW · 지휘자

어떻게 시키나

순서·상태. 언어만 바꿔 다시 써도 됨.

WHERE · 창구

어디서 받나

화면·채팅. 입구만 바뀌도 속은 그대로.

섞여 있으면 하나 바꾸려다 전부 건드린다. **가르면** 한 곳만 고쳐도 된다 — 그래서 안 무너진다.

도구를 어떻게 부르나

지휘자가 그림·영상 도구를 부른다 — 그 방식이 MCP와 API. 딱 두 줄만.

MCP

AI에 도구 꽂는 USB

붙이면 Claude가 바로 쓴다 · 지금 공장이 쓰는 방식

API

프로그램 주문 창구

정해진 양식으로 주문 → 정해진 답 · 무인 이식 후

SDK·REST·폴링 같은 말도 나오지만 — 오늘은 **이 두 줄**만 챙기면 충분.

MCP now → API later

같은 일을, 오늘은 MCP로 / 내일은 API로 부를 뿐. 설계는 안 바뀐다.

하는 일	오늘 · Cowork (MCP)	내일 · 무인 (API)
판단. 글쓰 기	Cowork 안의 Claude	Claude API
그림. 영상 생성	생성 도구 MCP	생성 도구 API(REST)
사람 승인	채팅으로 확인	웹 화면에서 승인

일(왼쪽)은 그대로, 부르는 방식(오른쪽)만 바뀐다 — 지휘자가 얇으니 가능.

비싼 길목엔 사람 관문

자동화라고 다 무인이 아니다. 되돌리기 어렵거나 비싼 단계 앞에 검수 관문.



특히 **스토리보드 검수** — 영상 만들기 직전이라 여기서 막으면 가장 큰 낭비를 막는다.

"어디에 브레이크를 달까"가 설계

관문을 다는 기준

- 그 뒤가 **비싼가?** (영상 생성처럼 돈·시간)
- 틀리면 **되돌리기 어려운가?** (발행·발송)
- 둘 다 예 → 그 앞에 관문

관문의 작동

- 자동 점검(체크리스트)
- → 사람 승인
- → 아니면 **직전 칸으로 되돌림**(반려 루프)

관문 0개 → 이상한 게 끝까지 간다. 너무 많음 → 사람이 계속 붙어 자동화 아님. **비싼 길목에만.**

파일이 곧 상태다

상태를 사람 머리·프로그램 메모리가 아니라 전부 파일로. 실행 1건 = 폴더 하나.

runs / 날짜_제품 / 폴더

- 중간 결과물·미디어를 전부 여기 쌓음
- 세션 끊겨도 → 다음 칸부터 이어서
- 이미 만든 건 건너뛸 (역등 · 재시작 안전)

그래서 — 갈아끼운다

MCP now → API later

상태가 파일에 다 있으니 지휘자만 바꾸면 무인 공장. 5차의 '기록'이 곧 T.

원칙 하나: "메모리에 숨은 상태 금지. 상태는 전부 파일에."

끝겨도 이어서 돈다

대본까지 만들고 노트북이 꺼졌다. 파일에 상태가 있으면 —

① 다시 켜다

폴더를 열어 "어디까지 됐나" 확인 — 대본 서류까지 있음

② 그림 단계부터 이어감

이미 있는 접수·분석·대본은 건너뛴 (역등)

③ 지휘자를 바꿔도 동일

Cowork로 시작했어도 Python이 같은 폴더 읽어 이어감

여기서 **PILOT 다섯 원칙이 하나로** 모인다 — 상태가 남으니 멈춰도, 갈아끼워도 굴러간다.

내 자동화 설계도 그리기

PILOT 5칸에 내 업무 하나를. 완성 못 해도 P·I만 채우면 절반은 성공.

글자	채울 것	한 줄 질문
P	단계로 쪼개기	내 업무를 3~7단계로?
I	입력→출력	각 단계 뭐 받아 뭐 내나?
L	지식·실행·창구	안 변하는 것 / 시키는 것?
O	사람 관문	꼭 확인할 지점은?
T	작업 폴더	무엇을 파일로 남기나?

예시 — "주간 리포트 자동화"

1
수집

2
분류

3
요약

4
작성

5
검토

I · 입력→출력

- 수집: 기간 → 기록 원본
- 요약: 묶음 → 3줄 요약
- 작성: 요약 → 리포트 초안

L · O · T

- L 지식=양식 / 실행=수집·요약 / 창구="이번 주 리포트"
- O 작성 뒤 사람 확인 + 최종 승인
- T 날짜_주제 폴더에 단계별 저장

자주 막히는 지점 — 이렇게 뚫는다

막힘 1

"단계를 몇 개로?" → 3~7개. 한 칸이 두 가지 일 하면 쪼갬다.

막힘 2

"입력·출력이 헛갈려요" → 앞 칸의 출력 = 내 칸의 입력인지 확인.

막힘 3

"관문을 어디에?" → 되돌리기 어렵거나 비싼 단계 앞 딱 한두 곳.

막힘 4

"다 못 채웠어요" → 괜찮다. P·I만 있으면 자동화의 뼈대는 완성.

설계도가 서면 절반은 끝 — 만드는 건 6·7차의 Claude Code 방식으로.

PILOT — 다섯 원칙

글자	원칙	한 줄	공장에서
P	Pipeline · 쪼갬다	큰 일을 순서 단계로 (가로)	접수→...→합본 8칸
I	I/O · 정한다	단계마다 입력→출력 계약	칸 사이 결과 파일 양식
L	Layer · 나눈다	지식·실행·창구 층 (세로)	지식/스펙/지휘/창구 4층
O	Oversight · 지킨다	비싼 지점 앞 사람 관문	대본·스토리보드·영상·최종
T	Trace · 남긴다	상태를 전부 파일로	runs / 날짜_제품 /

뼈대 P·L · 관절 I · 브레이크 O · 피 T

자동화는 코드가 아니라 설계다

"잘 쪼개고 잘 이으면, 지휘자는 갈아끼우면 된다." — 여러분 설계도가 그 시작.

1차
프롬프트

2차
지침 I/O

5차
자동화 이해

6차
도구 만들기

7차
앱 만들기

8차
자동화 설계

1·2차의 프롬프트·계약, 5차의 자동화, 6·7차의 만들기 — 오늘 **설계로** 묶였다.

THANK
YOU

고맙습니다 · Q&A

해부해서 원리를 얻었다 — 이제 어떤 자동화든 PILOT로 설계할 수 있다.